

PEB 结构深度解析与内存定位（完整知识点整理）

- 核心目标

掌握通过 `gs:[0x60]` 访问 `TEB` → `PEB`，遍历 `InMemoryOrderModuleList` 双向链表，手工获取 `kernel32.dll` 和 `ntdll.dll` 基址的方法，完全不依赖 `GetModuleHandle`。

- 涉及的核心结构体（按内存布局排列）

`_UNICODE_STRING`（Windows 内核通用字符串结构）

```
typedef struct _UNICODE_STRING {
    USHORT Length;           // 字符串当前长度（字节数，不含结尾 '\0'）
    USHORT MaximumLength;    // 缓冲区总容量（字节数）
    PWSTR Buffer;           // 指向宽字符数组的指针
} UNICODE_STRING;
```

> 注意：Length 是字节数，转换成字符数需 `Length / 2`（宽字符 `wchar_t` 占 2 字节）。

`_LIST_ENTRY`（双向链表节点）

```
typedef struct _LIST_ENTRY {
    struct _LIST_ENTRY Flink; // 指向下一个节点（Forward Link）
    struct _LIST_ENTRY Blink; // 指向上一个节点（Backward Link）
} LIST_ENTRY;
```

Flink：前进方向（从头部向后遍历时使用）

Blink：后退方向

`_PEB` 关键字段（进程环境块）

```
typedef struct _PEB {
    BYTE Reserved1[0x18]; // 0x00~0x17 保留/填充
    PPEB_LDR_DATA Ldr;    // 0x18 ← 指向加载器数据结构的指针
    // ... 后续字段省略
} PEB;
```

`_PEB_LDR_DATA` 关键字段（加载器数据）

```
typedef struct _PEB_LDR_DATA {
    ULONG Length; // 0x00
    BOOLEAN Initialized; // 0x04
    PVOID SsHandle; // 0x08
    LIST_ENTRY InLoadOrderModuleList; // 0x10
    LIST_ENTRY InMemoryOrderModuleList; // 0x20 ← 我们遍历的链表头
    LIST_ENTRY InInitializationOrderModuleList; // 0x30
} PEB_LDR_DATA;
```

`_LDR_DATA_TABLE_ENTRY` 关键字段（代码中的 `MY_LDR_DATA_TABLE_ENTRY`）

```
struct MY_LDR_DATA_TABLE_ENTRY {
```

```

LIST_ENTRY InLoadOrderLinks;           // 0x00
LIST_ENTRY InMemoryOrderLinks;         // 0x10 ← 链表节点（指向前后模块）
LIST_ENTRY InInitializationOrderLinks; // 0x20
PVOID DllBase;                         // 0x30 ← 模块基址（核心输出）
PVOID EntryPoint;                      // 0x38
ULONG SizeOfImage;                     // 0x40
UNICODE_STRING FullDllName;            // 0x48 完整路径
UNICODE_STRING BaseDllName;            // 0x58 文件名（如 L"kernel32.dll"）
// 后续字段省略（不影响遍历）
};

```

关键字段说明：

> DllBase：模块在进程内存中的起始地址。

> BaseDllName：模块名称（不含路径），用于匹配目标 DLL。

● 涉及的关键宏

CONTAINING_RECORD（核心宏>>>反推结构体首地址）

```
CONTAINING_RECORD(pEntry, MY_LDR_DATA_TABLE_ENTRY, InMemoryOrderLinks)
```

作用：通过结构体成员的地址反推整个结构体的起始地址。

数学原理：结构体首地址 = 成员地址 - 成员在结构体内的偏移量

实现原理（宏展开）：

```

((MY_LDR_DATA_TABLE_ENTRY)((PCHAR)(pEntry) -
(ULONG_PTR)&((MY_LDR_DATA_TABLE_ENTRY)0)->InMemoryOrderLinks)))

```

➤ (MY_LDR_DATA_TABLE_ENTRY)0：假设地址 0 处有一个结构体。

➤ &(...)->InMemoryOrderLinks：取该字段在地址 0 处的偏移量。

➤ (PCHAR)(pEntry) - 偏移量：计算出真正的结构体起始地址。

__readgsqword（编译器内联函数）

```
uintptr_t pPeb = __readgsqword(0x60);
```

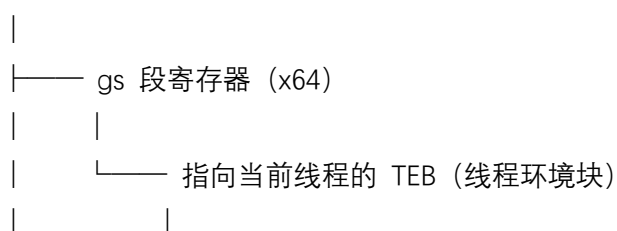
作用：读取 gs 段寄存器偏移 0x60 处的 8 字节（QWORD）数据。

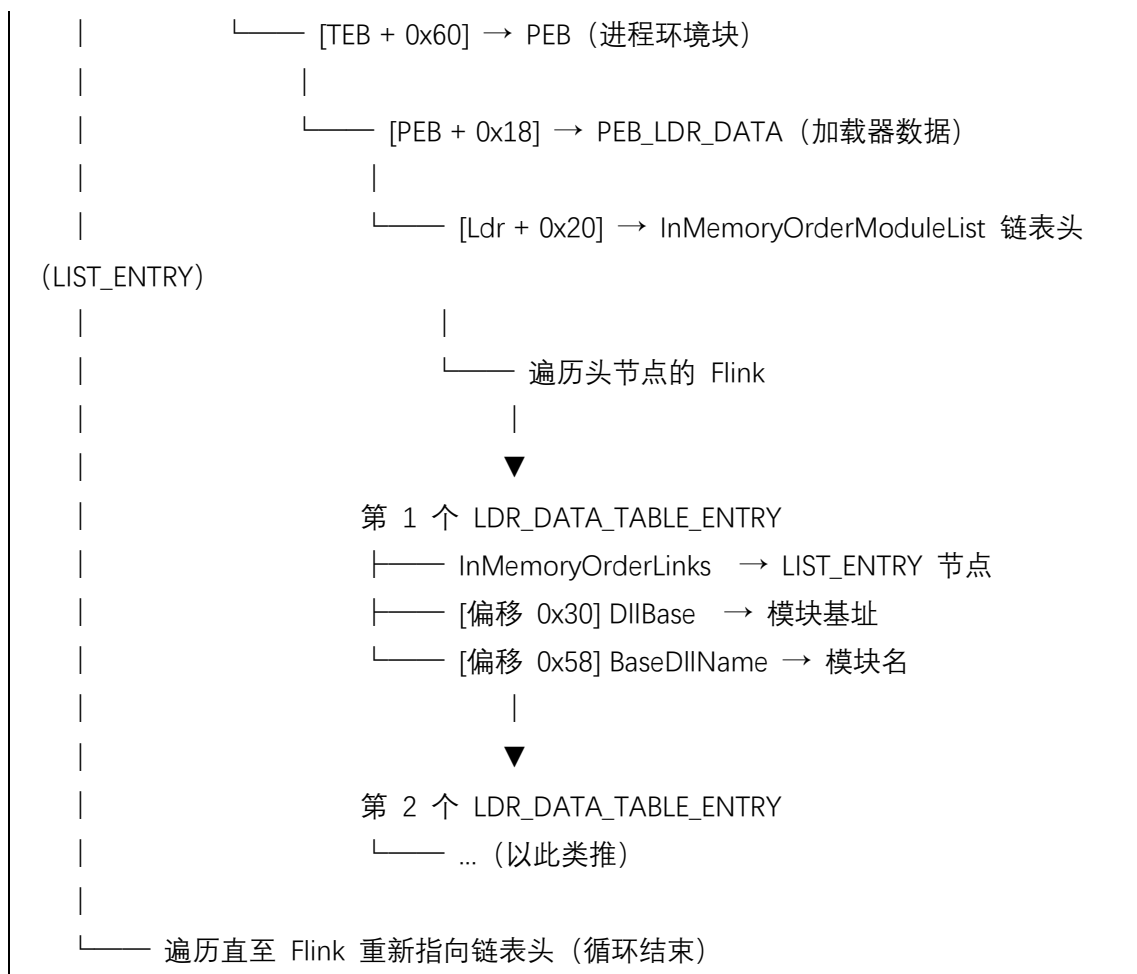
为什么是 gs：x64 下，gs 段寄存器固定指向当前线程的 TEB（线程环境块）。

为什么是 0x60：TEB 偏移 0x60 处存储着当前进程的 PEB 指针。

● 完整的利用链（内存寻址链路图）

硬件层





关键步骤文字总结：

- `gs:[0x60]` → 拿到 PEB 指针。
- `PEB + 0x18` → 拿到 Ldr (PEB_LDR_DATA)。
- `Ldr + 0x20` → 拿到 InMemoryOrderModuleList 的头部 (这是个 LIST_ENTRY 结构)。
- `ListHead->Flink` → 取出第一个实际模块的 LIST_ENTRY 节点。
- CONTAINING_RECORD → 通过该 LIST_ENTRY 节点，反推出完整的 LDR_DATA_TABLE_ENTRY。
- 读取 `Entry->DIIBase` (偏移 0x30) 获取基址；读取 `Entry->BaseDIIName.Buffer` (偏移 0x58) 获取模块名。
- 继续 Flink 遍历，直到回到链表头。

● 核心偏移量速查表 (Windows 10/11 x64)

描述	表达式	偏移值 (HEX)
PEB 指针在 TEB 中的位置	<code>gs:[0x60]</code>	0x60
Ldr 指针在 PEB 中的位置	<code>PEB + 0x18</code>	0x18
InMemoryOrderModuleList 在 Ldr 中的位置	<code>Ldr + 0x20</code>	0x20
DIIBase 在 LDR_DATA_TABLE_ENTRY 中的位置	<code>Entry + 0x30</code>	0x30
FullDIIName 在 LDR_DATA_TABLE_ENTRY 中的位置	<code>Entry + 0x48</code>	0x48
BaseDIIName 在 LDR_DATA_TABLE_ENTRY 中的位置	<code>Entry + 0x58</code>	0x58

> 特别提醒：这些偏移量在 Windows 10 22H2 / Windows 11 22H2 x64 上长期稳定。但不同 Windows 版本 (如 Win7、

Win8) 可能存在细微差异 (偏移 $\pm 0x8$)。实际工程中建议使用 `winternl.h` 官方定义或动态获取 `RTL_CONSTANT`，但为了学习原理，手工偏移是必经之路。

● 其他必要的扩展知识点

为什么 `gs` 段寄存器能直接访问？

- `x64` 的 `gs` 段寄存器在 CPU 层面有特权级区分 (内核态与用户态)。
- Windows 内核在创建线程时会把当前线程的 TEB 线性地址加载到 `gs` 基址寄存器中。
- 用户态代码通过 `gs:[偏移]` 访问 TEB/PEB，无需调用 API，速度快且不产生任何系统调用 (`syscall` 指令)，因此非常隐蔽。

遍历时 `Flink` 与 `Blink` 的方向

- 代码中使用 `pEntry = pEntry->Flink`，是从链表头部向后遍历。
- `Flink` 指向当前节点的下一个节点 (Forward Link)。
- 如果 `Flink == &ListHead` (或 `== pListHead`)，说明已回到头部，遍历结束。

`UNICODE_STRING::Length` 与 `Length / 2` 的陷阱

- `Length` 是字节数，但 `std::wstring` 构造需要字符数。
- 所以代码中写作：`std::wstring dllName(pName->Buffer, pName->Length / 2);`
- 若忘记除以 2，字符串会多出一倍长度，造成乱码或越界。

为什么 `DllBase` 可能为 0？

- 在某些极端情况下，未完全初始化或已卸载的模块节点可能残留，`DllBase` 为 `NULL`。
- 代码中通过 `if (dllBase == 0) { pEntry = pEntry->Flink; continue; }` 安全跳过。

● 对比 `GetModuleHandle` 的优势与劣势

维度	手工 PEB 遍历	<code>GetModuleHandle</code>
IAT 痕迹	无任何导入记录	会出现在导入表中，被杀软监控
调用开销	纯内存读取，无系统调用	进入内核查询，有 API 调用开销
可移植性	依赖偏移量，需适配版本	由系统保证兼容，无版本风险
难度	较高 (需理解结构体)	极低 (一行 API)
隐蔽性	完全静默	极易被钩子监控

`x86 (32 位)` 与 `x64 (64 位)` 的差异提醒

- `x64`: `gs:[0x60]` 指向 PEB。
 - `x86`: `fs:[0x30]` 指向 PEB。
 - 结构体内部偏移量也不同 (如 `Ldr` 在 `x86` 的 PEB 中偏移为 `0x0C`)。
 - 本课程及代码均基于 `x64` 环境，开发时项目属性务必选 `x64`。
- 本课代码里的“间接调用链”映射到硬件

CPU 寄存器 (`gs`)



└── 读取 `gs:[0x60]` → 得到 PEB 虚拟地址 (例如 `0x00007FFE0000`)

└—— 读取 PEB+0x18 → 得到 Ldr 虚拟地址 (例如 0x00007FFE1000)
|
└—— 读取 Ldr+0x20 → 得到链表头 ListHead 地址
|
└—— 读取 ListHead->Flink → 获取第一个节点指针
|
└—— 通过 CONTAINING_RECORD 得到 LDR_DATA_TABLE_ENTRY 起始地址
|
└—— 读取 +0x30 得到 DllBase
└—— 读取 +0x58 得到 BaseDllName.Buffer

● **必掌握的核心代码片段（手写关卡）**

- 获取 PEB: `uintptr_t pPeb = __readgsqword(0x60);`
- 获取 Ldr: `uintptr_t pLdr = (uintptr_t)(pPeb + 0x18);`
- 获取链表头: `LIST_ENTRY pListHead = (LIST_ENTRY)(pLdr + 0x20);`
- 遍历循环: `while (pEntry != pListHead) { ... pEntry = pEntry->Flink; }`
- 反推结构体: `CONTAINING_RECORD(pEntry, MY_LDR_DATA_TABLE_ENTRY, InMemoryOrderLinks)`